

ML Specification Report — playground-series-s5e10

Road Accident Risk Prediction • AIDE agent (20-step tree search, Claude Sonnet via shim)

Metrics, step counts, timings and every leaderboard figure are taken verbatim from the run's `facts_aide.json` (derived from `journal.json` for run `2-sassy-mouflon-of-virtuosity`). Pipeline description comes from AIDE's own auto-generated `report.md` and from `best_solution.py`; task metadata from `config.yaml`. Figures that exist only in a node's plan/analysis text are attributed inline.

Contents	
Road Accident Risk Prediction • AIDE agent (20-step tree search, Claude Sonnet via shim)	1
Overview	1
Data	2
Models & Architecture	3
Training procedures	4
Search trajectory	4
Specification fields	5
Inference procedures	6
Evaluation & Benchmarking	6

Overview

The task is to predict a continuous road accident-risk score in $[0, 1]$ from road and environmental features (metric: RMSE, minimize; target `accident_risk`). It is solved by **AIDE**, a tree-search coding agent that writes a complete solution script each step, executes it, reads back the printed CV score or the traceback, and mutates. Given a **20-step budget**, AIDE's champion appeared at **step 12** with **5-fold OOF RMSE 0.055994** — a three-way blend of a small-grid-tuned LightGBM, a CatBoost and a one-hot XGBoost, with blend weights found by simplex grid search on the out-of-fold predictions.

That submission scored **Public LB 0.05555 / Private LB 0.05579**, placing **862 / 4083 — the 78.9th percentile**, AIDE's strongest placement in this batch. On the private split, however, it finishes **second of three lanes**: our from-scratch agent at 0.05576, AIDE at 0.05579, NVIDIA at 0.05588. Both `local_winner` and `lb_winner` are recorded as **mine**.

Why it matters. Quantifying accident risk at the road-segment level is what turns a road-safety budget into a priority list — which curves get re-graded, which stretches get lighting, where a speed limit is

actually doing work. The dataset is a clean instance of a problem where the signal is nearly exhausted by any competent model, which makes it a useful stress test: it measures whether an agent knows when to stop, not whether it can find a trick.

Data

Purpose of Data. Predict a road segment's continuous accident-risk score — a regression problem where squared-error loss wants the conditional mean. **Data Format** is clean **tabular CSV** with no missing values, mixing numeric columns, booleans and low-cardinality categoricals. **Data Volume:** the competition config records **517K train rows / 173K test rows** with **12 predictive features** once `id` and the target are removed, and a target mean of 0.352.

Data Quality is high and unusually free of traps: the config records **no missing values** at all, so no imputation logic appears anywhere in the champion. The strongest signals recorded in the config are all consistent with road-safety intuition — **curvature dominant at $r \approx 0.54$** , **speed_limit second at $r \approx 0.43$** , and **lighting = night raising risk to ≈ 0.47 against ≈ 0.30 in daylight**. The real wrinkle is not data quality but **signal ceiling**: as the trajectory below shows, a plain LightGBM baseline lands within 0.000073 of the best result AIDE found in twenty steps.

Annotation Guidelines. The label is `accident_risk`, a continuous score bounded in $[0, 1]$; submissions are **real-valued predictions** scored by RMSE. The boundedness invites a link-function treatment, and AIDE tried one — a logit transform with sigmoid inversion at step 17 — which produced **0.056954**, the worst score of any non-degenerate node. Its report attributes the failure to the target not being a true probability with boundary mass, so the logit round-trip distorted the loss landscape relative to the RMSE the competition actually scores.

Feature Set. The champion models the **12 raw base features** directly, with the encoding varying by learner and **no engineered columns whatsoever**:

Group	Features
Categorical (4)	<code>lighting</code> , <code>road_type</code> , <code>time_of_day</code> , <code>weather</code>
Boolean, cast to int (4)	<code>holiday</code> , <code>public_road</code> , <code>road_signs_present</code> , <code>school_season</code>
Numeric (4)	<code>curvature</code> , <code>num_lanes</code> , <code>num_reported_accidents</code> , <code>speed_limit</code>
Engineered	none in the champion — interaction terms (step 2) and OOF-target-encoded categorical pairs were both tested and rejected

The encoding is deliberately per-model: LightGBM receives pandas `category` dtype and handles the four categoricals natively; CatBoost receives them as strings via `cat_features` indices for ordered target statistics; XGBoost receives a one-hot expansion built by `pd.get_dummies` over the concatenated train+test frames, which guarantees identical column sets across splits. Feature engineering was

attempted early and abandoned: explicit multiplicative interactions (`curvature × speed_limit` and friends) scored **0.056105** at step 2, worse than the untouched baseline's 0.056067, and AIDE's own report draws the right conclusion — GBDTs already approximate such products through splits, so writing them out adds columns without adding information.

Splitting strategy. A single **KFold(`n_splits=5`, `shuffle=True`, `random_state=42`)**, materialized once as `fold_indices` and reused identically for the LightGBM hyperparameter search, for all three base learners, and for the blend-weight grid — so the three models' OOF vectors are aligned row-for-row and can be blended honestly. The target is a continuous i.i.d. score with no time or group structure, so plain shuffled K-fold is the correct scheme.

There is one **methodological caveat worth stating plainly**: the same OOF partition is used three times over — to pick the best of six LightGBM configurations, to fit the three-way blend weights, and then to report 0.055994. Both selection layers optimize against the number being reported, so the champion metric is **selection-optimistic by construction**. **Leaderboard status: submitted and scored**, which lets us measure that optimism directly (see Evaluation), and the answer is reassuring — the private LB came in better than the local OOF, not worse.

Models & Architecture

Purpose of Architecture. A bounded continuous risk regressor minimizing RMSE. **Architecture Type** is a **three-member gradient-boosted decision-tree blend** — LightGBM (leaf-wise) + CatBoost (ordered boosting, symmetric trees) + XGBoost (depth-wise, histogram) — combined by a convex weighting found by exhaustive simplex grid search on OOF predictions.

All three members take the same **Input Dimension — 12 base features** per row, a flat vector with no spatial or sequence structure — differing only in **Input Format** as described above (native categoricals / `string cat_features` / one-hot).

Architecture Description. The LightGBM member is the only tuned one: AIDE ran a six-candidate grid over `num_leaves ∈ {31, 63, 95, 127, 255}`, `min_data_in_leaf ∈ {20, 30, 50, 75, 100, 200}`, `feature_fraction ∈ [0.6, 0.9]`, `bagging_fraction ∈ [0.6, 0.9]` and `lambda_l1 / lambda_l2 ∈ [0, 2]`, each scored on the full 5-fold OOF RMSE, on a base of `objective="regression", metric="rmse", learning_rate=0.05, seed=42` with up to 1500 rounds in the search and 2000 in the final fit, early-stopped at 50. CatBoost is `CatBoostRegressor(iterations=2000, learning_rate=0.05, depth=8, loss_function="RMSE", early_stopping_rounds=50, random_seed=42)`. XGBoost is `objective="reg:squarederror", learning_rate=0.05, max_depth=8, subsample=0.8, colsample_bytree=0.8, min_child_weight=5, seed=42`, 2000 rounds, early-stopped at 50.

Model Complexity is trees × leaves rather than a dense parameter count: three families × five fold-models each, capped at 1500–2000 boosting rounds with the realized counts set by early stopping (**not recorded**). The blend layer is a two-dimensional grid over the 3-simplex at **step 0.02**, minimizing OOF RMSE; per the step-12 node analysis the search settled on weights of roughly (**0.22 LGB**, **0.38 CatBoost**,

0.40 XGB) — a near-even three-way split, and the same analysis records all three members performing "similarly (~0.0560 RMSE individually)". That flatness is the whole competition in one sentence: no member dominates, so the blend can only buy variance reduction.

Training procedures

There is no hand-designed stage ladder. The training procedure is the search: 20 sequential nodes, each a full rewrite-execute-analyse cycle against the same 5-fold protocol.

Search trajectory

Of the 20 steps, **17 scored and 3 were buggy**, with `buggy_exc_types` recording **three `TimeoutError` s** and nothing else. Total execution was **10,265.7 s** (mean **513.3 s/node**, **max 1800.0 s** — the 30-minute per-node cap).

Step	OOF RMSE	Node (per AIDE's report)
0	0.056067	LightGBM baseline, native categoricals
1	0.07272	Ridge on one-hot + interactions
2	0.056105	explicit multiplicative interaction terms
3	0.05602	fixed 50/50 LightGBM + CatBoost blend
4	0.056018	grid-searched 2-way blend weight
5	0.057383	PyTorch MLP with categorical embeddings
6	0.056143	HistGradientBoostingRegressor
7	0.056037	XGBoost, one-hot, hist method
8	0.056019	Ridge stacking meta-learner over OOF preds
9	0.055998	3-way LGB + CatBoost + XGB simplex blend
10	0.055999	4-way blend adding ExtraTrees
11	0.055998	tuned CatBoost re-integrated into the 3-way blend
12	0.055994	grid-tuned LightGBM in the 3-way blend — champion
13	buggy	CatBoost hyperparameter search → <code>TimeoutError</code> at 1800 s
14	buggy	CatBoost hyperparameter search, retry → <code>TimeoutError</code> at 1800 s
15	0.056027	compute-constrained variant (fewer folds/rounds)
16	buggy	repeated 2×5-fold CV bagging → <code>TimeoutError</code> at 1800 s
17	0.056954	logit target transform
18	0.055998	3-way blend, restored
19	0.056062	compute-constrained variant

The entire 20-step search moved the metric by:

```
0.056067 - 0.055994 = 0.000073    (first scored node → champion)
0.000073 / 0.056067 = 0.0013      → 0.13% relative RMSE reduction
```

Seventy-three millionths of RMSE for twenty steps of search. That is the honest headline. Every gradient-boosted variant AIDE tried landed inside a band of roughly 0.0561–0.0560; the only nodes that moved appreciably were the ones that moved backwards — Ridge at 0.07272, the MLP at 0.057383, and the logit transform at 0.056954. AIDE's own report reaches the same conclusion, that "the dataset's signal is largely captured by any competent GBDT".

The timeout burn is the run's real failure mode. Three of twenty nodes died on the 30-minute cap, and because a timeout consumes the full budget before being killed, they were also the three most expensive nodes in the run:

```
3 × 1800.0 = 5400.0 s consumed by timeouts
5400.0 / 10265.7 = 0.5260    → 52.6% of all execution time produced no metric at all
```

Worse, the failures were **repetitive rather than exploratory**. Steps 13 and 14 propose essentially the same idea — a CatBoost hyperparameter search, never having tuned CatBoost — and both time out. The step-13 analysis diagnoses the cost correctly ("6 candidate configs × 5 folds × up to 2000 iterations ... far too expensive for the time budget") and prescribes the fix ("cut to 2–3 configs, reduce iterations, use fewer folds"), and then step 14 re-proposes a five-config search anyway and dies again, this time getting "through 3 of 5 configs before timeout". The agent wrote a correct post-mortem and did not act on it. Step 16 repeats the pattern with a different expensive design (repeated 2×5-fold CV across two seeds) and completes "only the first fold of the first repeat". The tree search's memory clearly carries metrics forward well — every plan cites the current best score — but it carries cost lessons forward poorly.

Post-peak behaviour is also weak. The champion landed at step 12, leaving seven steps; three of them died, and the four that scored (0.056027, 0.056954, 0.055998, 0.056062) all came in above 0.055994. Over a third of the budget after the peak produced nothing, and no stopping criterion intervened.

Specification fields

The **Loss Function** is squared error throughout — LightGBM `objective="regression"` with `metric="rmse"`, CatBoost `loss_function="RMSE"`, XGBoost `reg:squarederror` — so training loss, early-stopping criterion and competition metric are all the same quantity. The **Optimization Algorithm** is gradient boosting (histogram-based; leaf-wise for LightGBM, depth-wise for XGBoost, ordered/symmetric for CatBoost) — **not** SGD or ADAM — with blend weights found by exhaustive simplex grid search at step 0.02. The **Learning Rate** is the boosting shrinkage, **0.05** for all three members. There is **no Learning Rate Scheduler** (N/A — GBDT convergence is governed by 50-round early stopping on fold RMSE, not a schedule) and **no Batch Size** (N/A — full-dataset histogram boosting, not mini-batched).

Training Duration across the search was **10,265.7 s** — the most expensive run in this batch — at **mean 513.3 s** per node against a **1800.0 s** per-node cap, with 52.6% of it burned on timeouts as computed above. **Training Memory** is **not recorded**. **Transfer Learning** is N/A (no pretrained weights); the AIDE analogue is **within-run journal memory** — each plan explicitly names the incumbent ("Building on the established 3-way blend...") and proposes one atomic change. That memory does not cross competitions: AIDE starts each episode from the raw task description with no distilled priors, which is precisely why it re-derived from scratch that explicit interaction terms do not help GBDTs. **Data Augmentation** is N/A (tabular); the analogues attempted were feature engineering (rejected at step 2), target encoding of categorical pairs (rejected), and repeated-CV bagging (timed out at step 16).

Reproducibility Standards are moderate. `random_state=42` / `seed=42` is fixed in the fold splitter and in all three learners, and `fold_indices` is computed once and shared, so `best_solution.py` is re-runnable end to end. There is **no digit-for-digit rebuild gate** in the AIDE lane — the artifact of record is the emitted script, not a verified reconstruction. The doubled selection on the reporting OOF (hyperparameter choice, then blend weights) means 0.055994 should be read as an upper bound on the pipeline's true out-of-sample quality, not an unbiased estimate.

Inference procedures

Decision Threshold is N/A — RMSE is a squared-error regression metric, so raw real-valued predictions are submitted and never thresholded. **Post-processing** is a **clip to [0, 1]**, applied once to the final weighted blend, matching the target's known support; no rounding or snap-to-grid is used. Test predictions are built fold-wise — each of the three members accumulates `predict(X_test) / 5` across the five folds — and the three fold-averaged vectors are then combined with the OOF-fitted simplex weights before clipping. **Inference Duration** and **Inference Memory** are **not recorded**.

Evaluation & Benchmarking

Task for Performance Evaluation. Predict the test segments' accident-risk scores. **Performance Metrics.** RMSE (minimize) is the sole metric. AIDE's champion reaches **OOF RMSE 0.055994** at step 12, improving from a first scored node of 0.056067 — the 0.000073 / 0.13% gain computed above. Against the leaderboard:

$0.055994 - 0.05579 = 0.000204$	(local OOF - private LB; the private split scored BETTER than CV)
$0.05579 - 0.05555 = 0.00024$	(private LB - public LB)

The private leaderboard came in 0.000204 below the local OOF estimate — the pipeline generalized slightly better than its own cross-validation predicted, so the doubled selection on the OOF partition did not translate into a leaderboard shortfall. Note that this gap, 0.000204, is nearly three times the entire 0.000073 that twenty steps of search bought; the difference between the CV and LB scales dwarfs the difference between AIDE's first idea and its last.

Performance Benchmarking. All three agent lanes submitted to the same private split:

Agent	Approach	Private LB RMSE
Our from-scratch agent	GBDT pool + tree search	0.05576
AIDE	20-step tree search, 3-way GBDT blend	0.05579
NVIDIA	reproduce-agent lane	0.05588

$0.05579 - 0.05576 = 0.00003$
 $0.05588 - 0.05579 = 0.00009$

(our agent over AIDE)
(AIDE over NVIDIA)

Both recorded verdicts go the same way — `local_winner = mine`, `lb_winner = mine` — so our from-scratch agent takes this episode, by **0.00003 RMSE**. That margin should not be read as a capability difference; on a problem whose signal ceiling is this hard, three quite different search strategies landed inside a very narrow band:

$0.05588 - 0.05576 = 0.00012$
 $0.05602 - 0.055994 = 0.000026$

(full spread across all three lanes, private LB)
(AIDE's own step-3 blend → its step-12 champion)

AIDE's placement is nevertheless its best in the batch: **862 / 4083, the 78.9th percentile**. The fair summary is that AIDE performed well here and inefficiently — it had a competitive answer by step 3, spent nine more steps improving it by 0.000026, and then spent 5400 s discovering that its remaining ideas did not fit the compute budget.

Fields marked "not recorded": realized boosting-iteration counts after early stopping; training peak memory; inference duration and memory. One minor source discrepancy: AIDE's own `report.md` lists the step-10 four-way blend at 0.055998 and tabulates an ExtraTrees baseline of 0.057890 that does not appear in the journal-derived trajectory; the values in this report are those in `facts_aide.json` (step 10 = 0.055999), and the two sources are not reconciled here.